

Plum Claims Processing System

Architecture Document

Multi-Agent AI Pipeline · Health Insurance Automation



1. System Overview

This system automates health insurance claims processing using a **multi-agent pipeline architecture**. When an employee submits a claim with medical documents, the system orchestrates six specialised AI agents in sequence — validating documents, extracting data, checking policy rules, detecting fraud, making a decision, and assembling an auditable trace — all without human intervention.

Component	Role
Frontend (Nginx)	Vanilla HTML/CSS/JS dashboard — Submit Claims, View Traces
Backend (FastAPI)	POST /submit-claim → async job · GET /claim-status/{job_id} · GET /history
Multi-Agent Pipeline	Six specialised AI agents executed sequentially per claim
OCR Service	OpenAI Vision API (GPT-4o-mini) for images · pdfplumber for PDFs
Policy Config	policy_terms.json — all rules at startup, zero code changes needed
PostgreSQL DB	Persistent claim history and full audit traces via SQLAlchemy ORM

Claims Processing Pipeline

The diagram below illustrates the end-to-end flow of a claim through all six AI agents, showing the inputs, outputs, and decision points at each stage.

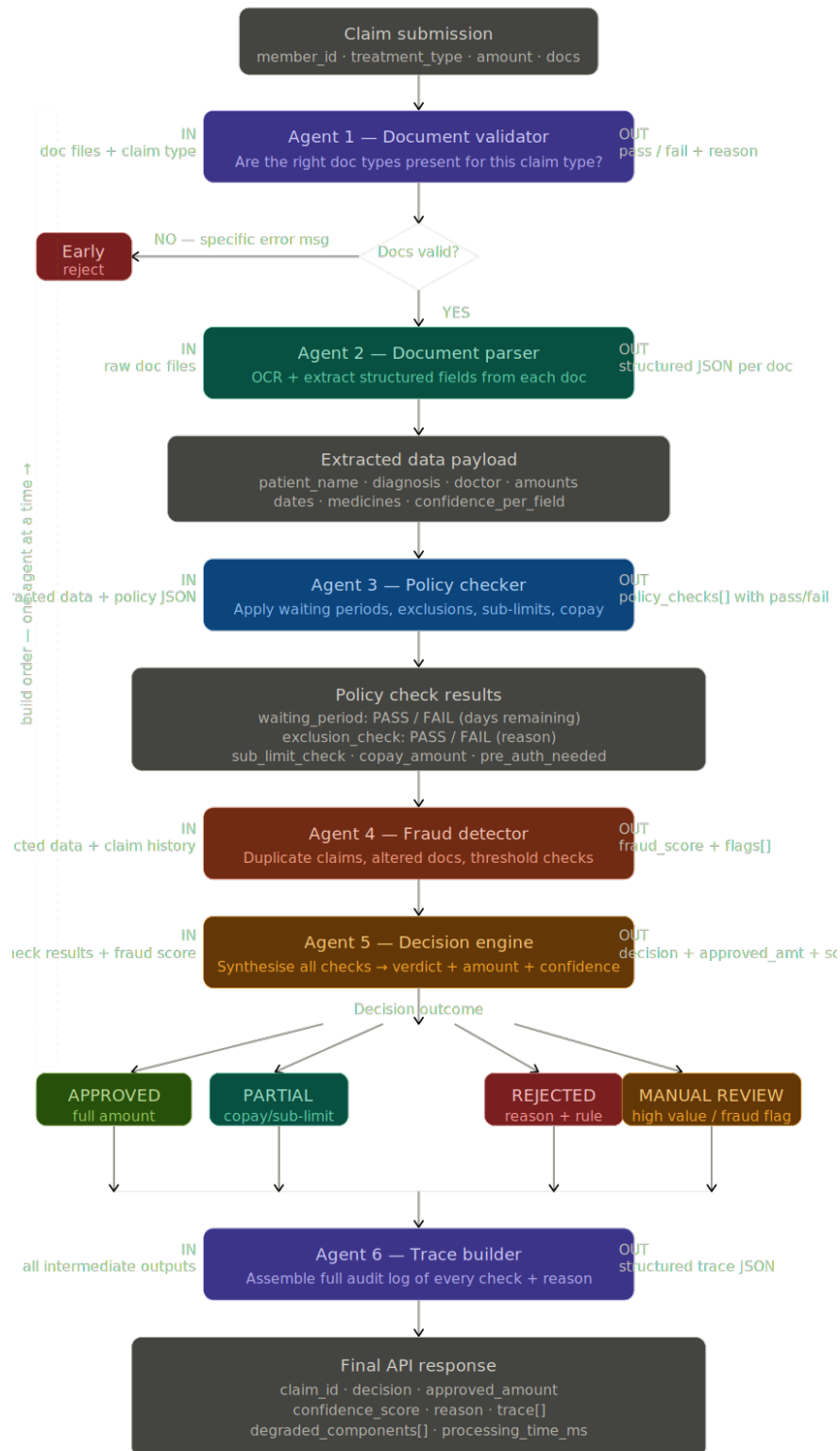


Figure 1 — End-to-end claims processing pipeline (6-agent architecture)

2. Component Breakdown

2.1 Frontend — Claims Dashboard

Vanilla HTML/CSS/JS served via Nginx in Docker. Handles claim submission, real-time pipeline animation (polling `/claim-status/{job_id}` every 500 ms), claims history with trace viewer, and aggregate dashboard statistics.

2.2 Backend — FastAPI Application

Python 3.12 / FastAPI / Uvicorn. Accepts multipart form submissions, manages async jobs via `asyncio.create_task + asyncio.to_thread`, and orchestrates the 6-agent pipeline. A shared **JOBS** dict (protected by `threading.Lock`) tracks per-job state.

2.3 OCR Service

OpenAI Vision API (GPT-4o-mini) for images, pdfplumber for PDFs. Vision was chosen over Tesseract because Indian medical documents are often handwritten, stamped, or poorly scanned — areas where vision models dramatically outperform traditional OCR. Trade-off: higher per-call cost for significantly better accuracy.

Agent Details

A1

Document Validator

Catches bad uploads early before spending compute on parsing.

GPT-4o-mini classifies each document (PRESCRIPTION, HOSPITAL_BILL, PHARMACY_BILL, LAB_REPORT, UNKNOWN), cross-checks against `policy_terms.json` requirements. Produces specific, actionable errors.

A2

Document Parser

Extracts structured JSON data from each validated document.

Type-specific prompts extract `patient_name`, `diagnosis`, `doctor`, `medicines`, `amounts`, `dates`, plus per-field confidence scores.

A3

Policy Checker

Applies all policy rules from `policy_terms.json` — zero hardcoded logic.

Checks: (1) 30-day initial + condition-specific waiting periods. (2) General and category exclusions. (3) Pre-auth for high-value tests >INR 10,000. (4) Network discount → copay → sub-limit financials.

A4

Fraud Detector

Flags anomalous claims for manual review before a verdict is reached.

Signals: same-day frequency ≥ 2 , duplicate (same date+amount), high value >INR 25,000, identity mismatch. Additive score capped at 1.0; score ≥ 0.8 triggers `MANUAL_REVIEW`.

A5

Decision Engine

Synthesises all upstream results into a final verdict.

Logic: `fraud_score` ≥ 0.8 → `MANUAL_REVIEW` · policy failure → `REJECTED` · approved < claimed → `PARTIAL` · otherwise → `APPROVED`. Confidence = $1.0 - (\text{fraud_score} \times 0.5)$.

A6

Trace Builder

Assembles a complete audit trail explaining exactly what happened.

Outputs JSON with claim summary and per-stage details from all five preceding agents: classifications, extracted fields, policy results, fraud flags, and final decision with reasoning.

3. Design Decisions & Trade-offs

Decision	Rationale
Multi-agent pipeline over monolith	Clean separation of concerns; each agent testable and replaceable independently.
GPT-4o-mini over GPT-4o	10x cheaper with comparable accuracy for classification and extraction tasks.
OpenAI Vision over Tesseract	Indian medical docs are messy. Vision models handle handwritten text dramatically better.
Async jobs + polling	LLM calls take 5-15 s; synchronous endpoints would time out. Polling keeps the API responsive.
Policy rules from JSON	Adding a category or changing a limit requires zero code changes.
PostgreSQL over SQLite	Production-grade persistence; handles concurrent writes; Render managed DB.
Vanilla JS over React	No build step; reduces deployment complexity for a dashboard UI.
Thread concurrency over Celery	asyncio.to_thread is sufficient at current scale; avoids Redis infrastructure overhead.

What We Rejected

- **LangChain / LangGraph** — Heavy abstraction over a linear pipeline. Plain Python is simpler and more debuggable.
- **SSE / WebSocket streaming** — 500 ms polling gives equivalent UX for a pipeline completing in ~10 s.
- **Microservice per agent** — Overengineered for current scale; agents share the same process, eliminating network overhead.

4. How Failures Are Handled

The system degrades gracefully rather than crashing:

Failure Scenario	Behaviour
LLM Timeout / API Error	Pipeline wraps processing in try/except; job status set to FAILED with human-readable message.
Bad Document (UNKNOWN)	Agent 2 skips with status SKIPPED; pipeline continues with remaining documents.
Database Failure	save_claim() catches exceptions internally; user still receives their claim result.
Missing API Key	Logged as warning at startup; agents return empty/default results rather than crashing.
Frontend Disconnection	Results stored in-memory (JOBS dict) and can be re-pollled at any time.

5. Scaling to 10x Load

If Plum scales from 75K to 750K claims per year, the following changes are required:

Area	Current	At 10x
Job Queue	In-memory dict + threading	Celery + Redis; horizontally scaled workers
Database	Single PostgreSQL instance	Read replicas; partition claim_history by date
LLM Calls	Sequential per claim	Parallelise Agent 1 across docs; batch API calls
Caching	None	Redis for policy_terms.json, member lookups, classifications
API	Single Uvicorn process	Load balancer + multiple Uvicorn workers (--workers N)
Fraud Detection	In-memory + DB history	Dedicated microservice with streaming event processing
Observability	Logging to stdout	Structured JSON logging, OpenTelemetry, Prometheus metrics

6. Deployment Architecture

All services are containerised with multi-stage Docker builds and deployed on Render:

Service	Type	Details
PostgreSQL Database	Managed DB	claim_history table; persistent across deployments
Backend Web Service	Docker Container	FastAPI + Uvicorn · env: DATABASE_URL, OPENAI_API_KEY · Port 8000
Frontend Static Site	Docker + Nginx	HTML/CSS/JS on port 80 · API_BASE points to backend URL